1. Project Overview

This project implements a real-time data ingestion and processing pipeline using Apache NiFi and Apache Hadoop HDFS. The system simulates an e-commerce environment where transaction data is continuously generated, processed, validated, cleaned, enriched, and stored in HDFS for big data analytics purposes.

The project demonstrates multiple ingestion techniques including:

- Real-time file ingestion
- Relational database ingestion
- API ingestion

The solution was implemented using Docker containers for portability and environment consistency.

2. Design Goals

The following goals guided the system design:

- Simulate realistic streaming transaction data
- Handle semi-structured and messy data
- Demonstrate enterprise-style ingestion pipelines
- Ensure scalability and reliability
- Apply Apache NiFi best practices
- Store processed data in distributed storage (HDFS)
- Separate valid, invalid, and duplicate data

3. Technology Stack

Technology	Purpose
Apache NiFi	Data ingestion and transformation
Apache Hadoop HDFS	Distributed storage
PostgreSQL	Relational database ingestion
Docker	Containerized environment
Python	Real-time data simulation
JSON	Semi-structured data format

4. Real-Time Data Simulation Design

A Python script was developed to simulate real-time e-commerce transactions. The script continuously generates JSON transaction files every few seconds using filenames formatted as:

transaction_YYYYMMDD_HHMMSS.json

This naming convention was selected because:

- it guarantees unique filenames,
- improves traceability,
- prevents accidental overwrites,
- resembles enterprise logging and ingestion systems.

The generated records intentionally include:

- missing values,
- invalid prices,
- inconsistent formats,
- duplicate transaction IDs.

This approach was chosen to simulate realistic dirty data scenarios commonly encountered in production systems.

5. File-Based Ingestion Design

The ingestion pipeline was implemented using:

ListFile → FetchFile

instead of using GetFile.

This design decision was made because:

- it follows enterprise ingestion best practices,
- separates file discovery from file retrieval,
- supports better scalability,
- improves reliability in clustered environments,
- reduces duplicate processing risks.

The ListFile processor continuously monitors the transaction directory and tracks already processed files using NiFi state management.

FetchFile retrieves the actual file content after successful discovery.

6. Data Transformation Design

The transformation phase was designed to improve data quality and separate invalid records from valid transactions.

Several processors were used:

Processor	Purpose
EvaluateJsonPath	Extract JSON fields into attributes
RouteOnAttribute	Validate records
ReplaceText	Clean invalid price formats
UpdateAttribute	Handle missing values

The following transformations were implemented:

- removal of $ symbols from price values,
- detection of missing email addresses,

- validation of invalid prices,
- handling missing quantity values.

Invalid transactions were routed into separate storage directories to ensure fault isolation and maintain pipeline continuity.

7. Duplicate Handling Design

Duplicate transaction detection was implemented using RouteOnAttribute.

Transactions containing the identifier:

ORD_DUPLICATE

were routed to a dedicated duplicate storage path.

This simplified duplicate detection approach was selected because:

- it is easier to maintain,
- avoids unnecessary infrastructure complexity,
- demonstrates duplicate handling logic clearly,
- is sufficient for educational purposes.

Separate handling of duplicate records improves data quality and prevents analytical inconsistencies.

8. Database Ingestion Design

Relational database ingestion was implemented using:

- DBCPConnectionPool
- QueryDatabaseTable

The PostgreSQL database contains:

- customers table,
- products table.

QueryDatabaseTable was selected instead of ExecuteSQL because it supports incremental extraction through maximum-value columns.

This design:

- avoids full table reloads,
- reduces unnecessary processing,
- improves scalability,
- demonstrates enterprise ETL practices.

The following maximum-value columns were used:

- customer_id
- product_id

Database output was converted from AVRO to JSON before storage.

9. API Ingestion Design

An additional ingestion technique was implemented using:

- InvokeHTTP

The pipeline consumes data from an external REST API using scheduled HTTP requests.

This design demonstrates:

- integration with external systems,
- real-time API ingestion,
- JSON response handling,
- validation of external data sources.

API responses were validated and separated into:

- clean API data,
- invalid API data.

10. HDFS Storage Design

Processed data was stored in HDFS using:

- PutHDFS

HDFS was selected because it supports:

- distributed storage,
- fault tolerance,
- scalability for big data systems.

The final HDFS structure was organized as:

/ecommerce

|

├── clean_transactions

├── invalid_transactions

├── db_customers

├── db_products

└── api_data

  ├── clean_data

  └── invalid_data

This directory structure was designed to:

- logically separate data sources,
- improve maintainability,
- simplify future analytics workflows.

11. Docker-Based Architecture Design

The entire environment was deployed using Docker containers.

Separate containers were used for:

- Apache NiFi,
- Hadoop/HDFS,
- PostgreSQL.

Docker was selected because it:

- simplifies environment setup,
- improves reproducibility,
- reduces dependency conflicts,
- enables isolated service management.

Container networking was configured to allow communication between NiFi and HDFS.

12. NiFi Best Practices Applied

Several Apache NiFi best practices were implemented throughout the pipeline.

Back Pressure

Queue thresholds were configured to prevent memory overflow and stabilize flow execution during high ingestion rates.

Yield Duration

Yield durations were configured to prevent continuous retry attempts during temporary failures.

Relationship Handling

All processor relationships were explicitly connected or auto-terminated to ensure proper flow management.

Modular Pipeline Design

The pipeline was divided into logical stages:

- ingestion,
- validation,
- cleaning,
- duplicate handling,
- storage.

This improves readability, maintainability, and debugging.

Failure Isolation

Invalid and duplicate records were routed into separate directories instead of interrupting the entire workflow.

13. Challenges Encountered

Several technical challenges were encountered during implementation.

HDFS Connection Issues

NiFi initially failed to connect to HDFS because localhost was incorrectly referenced inside Docker containers. This was resolved by using Docker container hostnames instead of localhost.

Processor Compatibility

Some older NiFi processors were unavailable in version 2.9.0. Alternative processors and simplified routing approaches were implemented to maintain compatibility.

Docker Networking

Container communication required proper Docker network configuration to allow interoperability between NiFi, PostgreSQL, and Hadoop containers.

14. Conclusion

This project successfully demonstrates a complete real-time data ingestion and processing pipeline using Apache NiFi and Hadoop HDFS.

The system integrates:

- file ingestion,
- database ingestion,
- API ingestion,
- data transformation,
- validation,
- distributed storage.

The implemented architecture follows modern ETL and big data engineering practices while maintaining scalability, reliability, and modularity.